

Arquitectura Técnica del Sistema de Gestión Departamental

Sistema de Gestión Departamental de Informática y Computación
(SGDIC)

Proyecto: Sistema de Gestión Departamental

Área: Informática y Computación

Versión: 1.0

Página: 1

Documento técnico que describe la arquitectura de software
del SGDIC: frontend, backend en Django, base de datos y despliegue.

Índice general

Acrónimos y Abreviaturas	1
0.0.1. Organizaci3n por roles	2
1. Arquitectura de Software del Sistema	3
1.1. Capa de Presentaci3n (Frontend)	4
1.1.1. Tecnolog3as y lenguajes	4
1.1.2. Organizaci3n por roles	4
1.2. Capa de Aplicaci3n (Backend con Django)	5
1.2.1. Lenguaje y estructura	6
1.2.2. Organizaci3n por roles	6
1.3. Capa de Datos (PostgreSQL)	7
1.4. Comunicaci3n entre Componentes	8
1.5. Capa Anal3tica y Motor de Soluciones	9
1.6. Despliegue y Operaci3n	10
1.7. Seguridad y Control de Acceso	10
Referencias	12

Acrónimos y Abreviaturas

SGDIC	Sistema de Gestión Departamental de Informática y Computación
UML	Unified Modeling Language (Lenguaje Unificado de Modelado)
DJ	Django
ORM	Object-Relational Mapping (Mapeo Objeto-Relacional)
API	Application Programming Interface
REST	Representational State Transfer
RBAC	Role-Based Access Control
SQL	Structured Query Language
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
WSGI	Web Server Gateway Interface
ACID	Atomicity, Consistency, Isolation, Durability
ETL	Extract, Transform, Load (Extracción, Transformación y Carga)
KPI	Key Performance Indicator (Indicador Clave de Desempeño)
CI	Integración Continua
CD	Despliegue Continuo
HTTPS	HyperText Transfer Protocol Secure
JSON	JavaScript Object Notation
SPA	Single Page Application (Aplicación de Página Única)
MVT	Model-View-Template (Modelo-Vista-Plantilla)

0.0.1 Organización por roles

El backend filtra la información por rol (Role-Based Access Control ([RBAC](#))) de forma transversal: la autenticación y los permisos se gestionan en la app **usuario**, y cada app aplica `get_queryset` filtrado por el usuario autenticado. Así, el rol **estudiante** interactúa con **cupos**, **mensajes** y **evaluaciones**; el rol **docente** aporta a **materias**, **evaluaciones** (asistencia, justificaciones, resultados) y **mensajes**; y la **secretaría** accede a **reportes** (listas de espera, Key Performance Indicator (Indicador Clave de Desempeño) ([KPI](#))) y **quejas**, con intervención trazada en hilos escalados. El **departamento** configura parámetros globales (preguntas de evaluación, [KPI](#)) y la app **analitica** consolida datos para el motor de soluciones rápidas.

La figura [1.4](#) muestra la arquitectura del backend: el servidor de aplicaciones de Django ([DJ](#)) atiende peticiones Representational State Transfer ([REST](#)) desde el frontend, persiste en la base de datos y alimenta al motor analítico de forma asíncrona. »

Capítulo 1

Arquitectura de Software del Sistema

El Sistema de Gestión Departamental de Informática y Computación (SGDIC) se diseñó bajo una arquitectura de tres capas (*presentación*, *aplicación* y *datos*) sobre la plataforma DJ. Esta organización separa claramente la interfaz de usuario, la lógica de negocio y el acceso a la información, facilitando mantenimiento, pruebas e incorporación de nuevos requerimientos del modelado Unified Modeling Language (Lenguaje Unificado de Modelado) (UML) documentado en el informe de planificación del proyecto.

La figura 1.1 muestra la vista general: cada capa se comunica con la inferior mediante Application Programming Interface (API)s de estilo REST, y la capa analítica se nutre de forma asíncrona mediante un motor de extracción Extract, Transform, Load (Extracción, Transformación y Carga) (ETL).

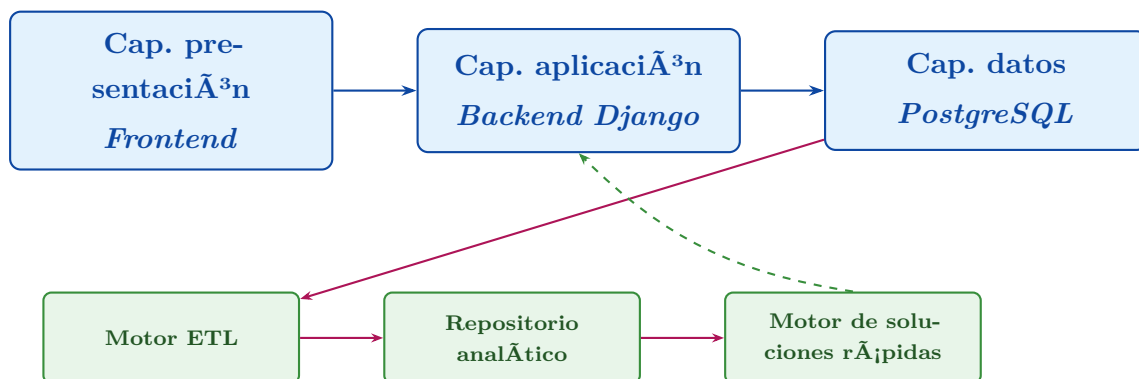


Figura 1.1: Vista de capas de la arquitectura del SGDIC

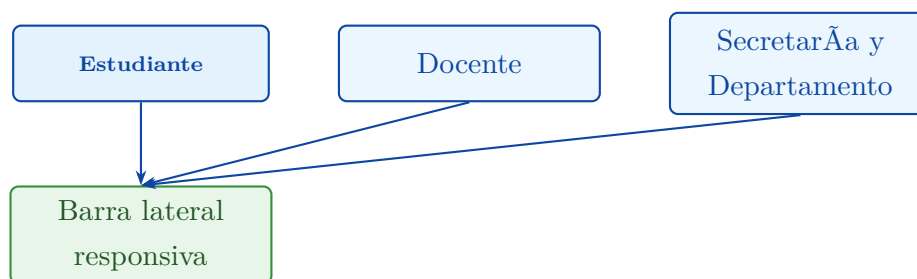


Figura 1.2: Roles y visibilidad de la interfaz del SGDIC

1.1 Capa de Presentaci3n (Frontend)

La capa de presentaci3n corresponde al *frontend* del SGDIC, construido con plantillas HyperText Markup Language (HTML) renderizadas por DJ y estilizadas con Cascading Style Sheets (CSS) (Bootstrap) y componentes interactivos en JavaScript (JS). La interfaz es responsiva y sirve tanto a estudiantes como a docentes, secretarĀa y departamento, adaptĀndose al rol autenticado (RBAC).

1.1.1 TecnologĀas y lenguajes

El cliente se construye con pila web estĀndar renderizada del lado del servidor (Model-View-Template (Modelo-Vista-Plantilla) (MVT)): HTML5 semĀntico para la estructura, CSS3 con Bootstrap 5 (*grid*, utilidades y componentes responsivos) para el estilo y JS (ES6+) para la interactividad ligera. El JS incluye jQuery para manipulaci3n DOM y llamadas asĀncronas a la API REST, y Bootstrap JS para componentes dinĀmicos (modales, tooltips, off-canvas). No se adopta arquitectura Single Page Application (Aplicaci3n de PĀgina Űnica) (SPA); las vistas se renderizan en el servidor y el navegador recibe marcado ya armado, lo que favorece el indexado y la carga inicial, manteniendo compatibilidad con entornos con soporte limitado.

1.1.2 Organizaci3n por roles

La interfaz se muestra a partir de los tres roles definidos en el documento de planificaci3n (figura 1.2):

Estudiante portal de cupos en tiempo real, confirmaci3n/rechazo de horarios, presentaci3n de cuestionarios, visualizaci3n de faltas y justificaciones, mensajerĀa con docentes y

autoevaluación histórica.

Docente gestión de materias y horarios, registro rápido de asistencia (lista o código), aprobación/rechazo de justificaciones, envío de mensajes masivos e individuales, consulta de resultados consolidados y comparativo histórico.

Administrador/departamento panel de listas de espera, intervención en hilos escalados, tableros consolidados de KPI y exportación de reportes a **CSV!** (CSV!) y **PDF!** (PDF!).

La adaptación se realiza mediante **RBAC**: la barra lateral y el menú navegacional sólo muestran los ítems autorizados al rol autenticado, y cada app del backend filtra su queryset según el usuario. La misma plantilla se reutiliza para todos los roles; el contenido visible depende del contexto que el backend inyecta a la vista.

La figura 1.3 muestra la arquitectura del cliente: el navegador consume las vistas renderizadas del servidor y, para interacciones en tiempo real, delega en notificaciones push y en la API REST del backend. Los recursos estáticos (css, js, imágenes) se entregan comprimidos y con caché mediante la configuración de archivos **STATICFILES** y **Whitenoise** (o CDN en despliegue productivo).

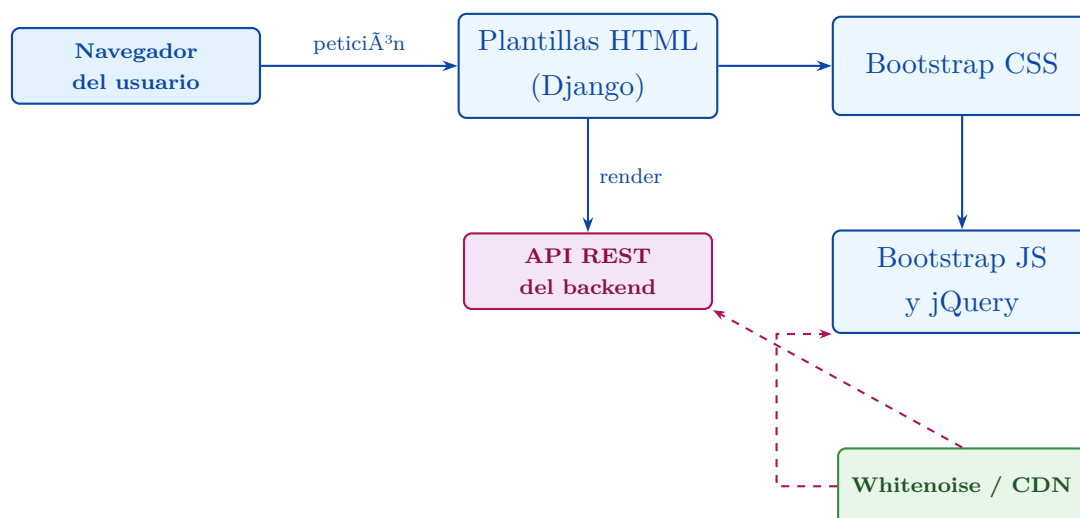


Figura 1.3: Arquitectura del frontend del SGDIC

1.2 Capa de Aplicación (Backend con Django)

El *backend* del SGDIC se implementa con **DJ**, un framework web de alto nivel escrito en Python que provee una arquitectura de aplicación por componentes. El proyecto se

organiza en apps modulares que agrupan las funcionalidades definidas en los requerimientos del Capítulo 4 del documento de planificación. Cada app contiene su propio paquete de `models`, `views`, `forms`, `urls` y pruebas, y se comunican entre sí a través del ORM de [DJ](#) y de rutas [REST](#) bien definidas.

Las apps principales son:

- usuario** Autenticación, roles ([RBAC](#)) y recuperación de cuenta.
- cupos** Gestión de disponibilidad, prerequisites y prioridad.
- materias** Catálogo, versiones y flujo de aprobación de materias.
- mensajes** Mensajería trazable y notificaciones automáticas.
- evaluaciones** Cuestionarios, evaluación docente y registro de faltas.
- quejas** Recepción, clasificación y trazabilidad de quejas.
- reportes** Tableros y exportaciones para secretaría y departamento.
- analitica** Extracción [ETL](#), [KPI](#) y motor de soluciones.

1.2.1 Lenguaje y estructura

El backend se escribe en **Python 3.11+** sobre [DJ 4.x](#), siguiendo el patrón [MVT](#) (*Model-View-Template*): los `models` definen el esquema de la base de datos, las `views` encapsulan la lógica de negocio y los `urls` enrutan las peticiones [REST](#). Las vistas retornan respuesta JavaScript Object Notation ([JSON](#)) a la API o renderizan plantillas [HTML](#) al frontend, según corresponda. El ORM de [DJ](#) traduce automáticamente las clases del modelo a consultas Structured Query Language ([SQL](#)), previniendo inyección y centralizando las reglas de integridad en migraciones versionadas.

1.2.2 Organización por roles

El backend filtra la información por rol ([RBAC](#)) de forma transversal: la autenticación y los permisos se gestionan en la app `usuario`, y cada app aplica `get_queryset` filtrado por el usuario autenticado. Así, el rol **estudiante** interactúa con `cupos`, `mensajes` y `evaluaciones`; el rol **docente** aporta a `materias`, `evaluaciones` (asistencia, justificaciones, resultados) y `mensajes`; y la **secretaría** accede a `reportes` (listas de espera, [KPI](#)) y `quejas`, con intervención trazada en hilos escalados. El **departamento** configura

parámetros globales (preguntas de evaluación, [KPI](#)) y la app `analitica` consolida datos para el motor de soluciones rápidas.

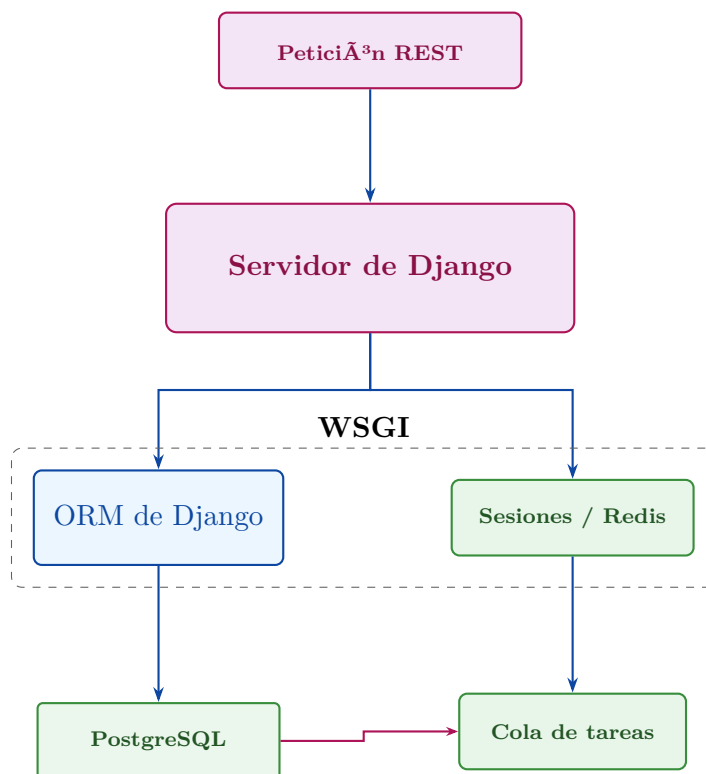


Figura 1.4: Arquitectura del backend de Django del [SGDIC](#)

1.3 Capa de Datos (PostgreSQL)

La capa de datos se soporta en PostgreSQL, un sistema gestor de bases de datos relacionales de código abierto con soporte transaccional Atomicity, Consistency, Isolation, Durability ([ACID](#)), tipos de datos avanzados y alto desempeño en cargas concurrentes. La elección responde a tres necesidades del [SGDIC](#): integridad referencial estricta para el flujo de cupos y matrículas, consultas analíticas sobre históricos de varios semestres y capacidad de crecimiento horizontal mediante réplicas de lectura.

El acceso a la base de datos se realiza exclusivamente a través del Object-Relational Mapping (Mapeo Objeto-Relacional) ([ORM](#)) de [DJ](#), que traduce las clases del modelo a tablas y relaciones. Esta decisión elimina la construcción manual de [SQL](#), previene inyección de código y concentra las reglas de integridad en las migraciones versionadas del proyecto. Las operaciones de escritura se agrupan en transacciones atómicas (`atomic`)

para garantizar que un cambio de cupo y su registro de auditoría se confirmen o se reviertan en conjunto.

La figura 1.5 presenta el modelo físico simplificado. Las entidades centrales son el usuario y su rol, la asignatura con sus versiones y prerequisites, la oferta de cupos por periodo, la matrícula, la mensajería, la evaluación y la queja. Las tablas de auditoría y de eventos analíticos cierran el modelo: la primera registra toda modificación sensible y la segunda alimenta el repositorio analítico descrito en la Sección 1.5.

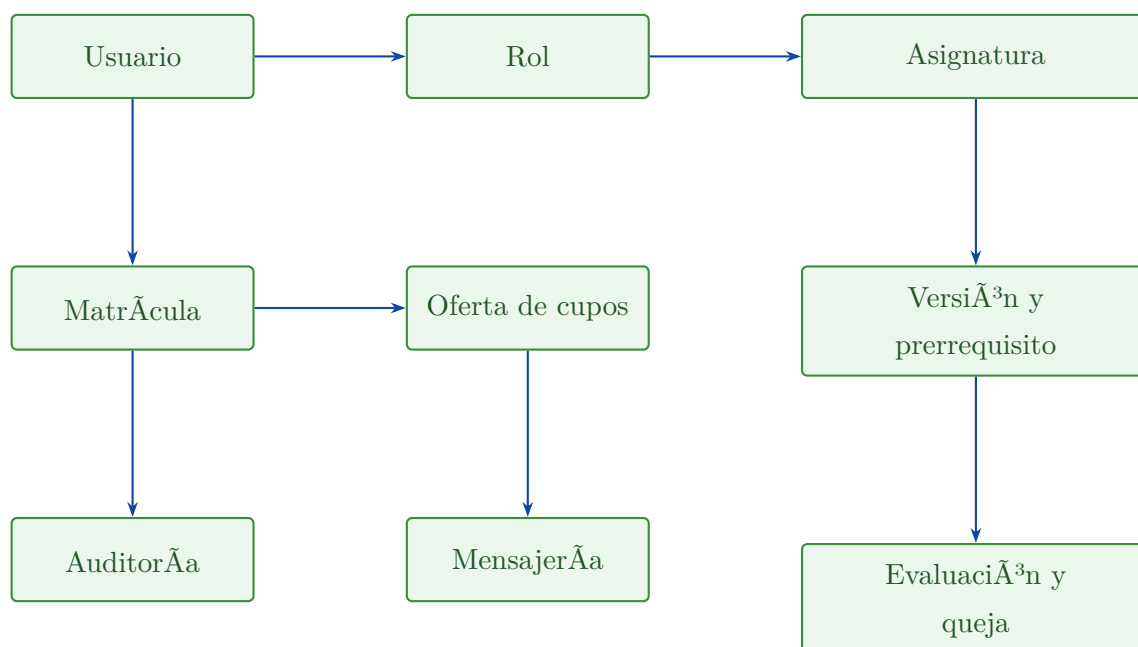


Figura 1.5: Modelo físico simplificado de la base de datos del SGDIC

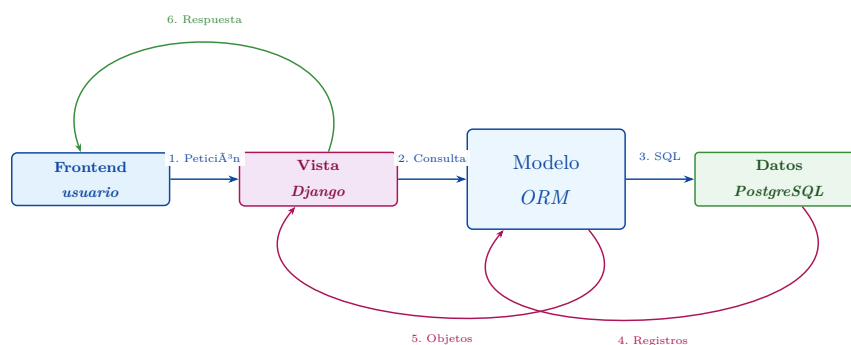
El esquema se administra con migraciones de [DJ](#) versionadas en el repositorio, se respalda diariamente con `pg_dump` y las consultas más frecuentes cuentan con índices sobre llaves foráneas, periodos académicos y estados de trámite.

1.4 Comunicación entre Componentes

La comunicación sigue el principio de dependencia unidireccional: la capa de presentación consume a la de aplicación y esta, a la de datos. El navegador solicita una vista; el servidor de [DJ](#) la resuelve a través de `urls`, `views` y `templates`, consulta el [ORM](#) y devuelve [HTML](#) renderizado o un cuerpo [JSON](#). Las operaciones que requieren actualizar la pantalla

sin recargar (notificaciones, validaci3n de prerrequisitos, filtros de cupos) usan llamados as3ncronos a la API REST con **JSON**.

La figura 1.6 resume el ciclo completo de una solicitud t3pica, desde que el usuario env3a el formulario hasta que recibe la respuesta, incluyendo las tareas que se delegan a la cola de procesos en segundo plano.



Reglas de la capa de aplicaci3n: validaci3n de permisos por rol, transacci3n at3mica para toda escritura, registro en auditor3a y notificaci3n as3ncrona al interesado mediante la cola de tareas.

Figura 1.6: Secuencia de comunicaci3n entre las capas del SGDIC

1.5 Capa Anal3tica y Motor de Soluciones

La capa anal3tica complementa las tres capas funcionales. Un proceso **ETL** programado extrae de forma peri3dica los eventos relevantes (matr3culas, cancelaciones, reclamaciones, respuestas de evaluaci3n), los normaliza y los carga en un repositorio anal3tico separado. Sobre ese repositorio se calculan los **KPI** del departamento y se alimenta el motor de soluciones r3pidas, que sugiere alternativas ante situaciones recurrentes, por ejemplo una asignatura con demanda superior a la oferta o un incremento sostenido de quejas por periodo.

El aislamiento del repositorio anal3tico evita que las consultas de reporte compitan con las transacciones acad3micas y permite recalcular indicadores sin afectar el servicio.

1.6 Despliegue y Operación

El sistema se despliega con el patrón de servidor de aplicaciones Web Server Gateway Interface (WSGI) detrás de un servidor web inverso que termina HyperText Transfer Protocol Secure (HTTPS), sirve los archivos estáticos y canaliza las peticiones dinámicas. La base de datos PostgreSQL se ejecuta como servicio independiente con acceso restringido por red. La figura 1.7 presenta la topología propuesta.

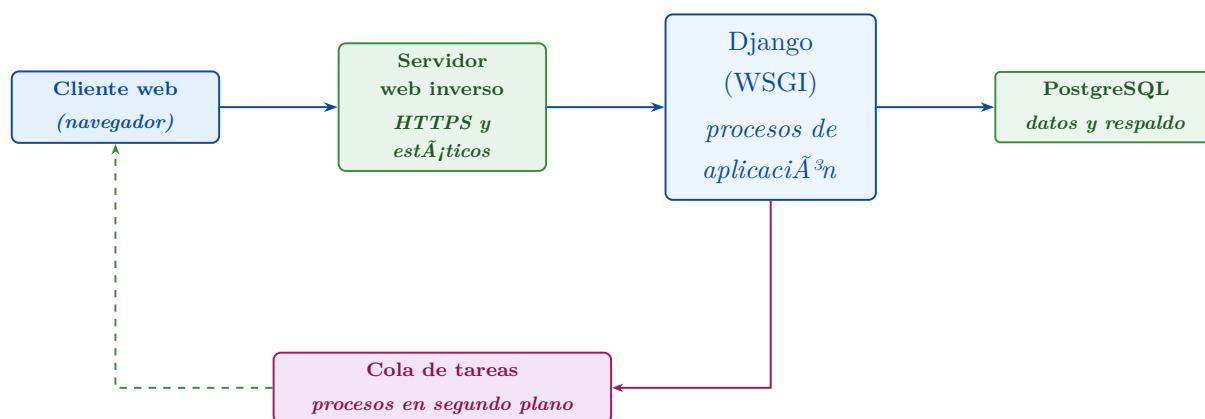


Figura 1.7: Topología de despliegue del SGDIC

La operación prevé integración y despliegue continuos (Integración Continua (CI) y Despliegue Continuo (CD)): cada entrega ejecuta las pruebas automatizadas, aplica las migraciones de base de datos y publica la versión en el entorno de producción, con posibilidad de reversión ante fallos. Los respaldos diarios, las migraciones versionadas y los índices sobre las consultas críticas constituyen el plan de contingencia y continuidad del servicio.

1.7 Seguridad y Control de Acceso

La seguridad se apoya en cuatro medidas. Primero, autenticación obligatoria con contraseñas almacenadas mediante funciones de derivación de clave y sesiones con expiración. Segundo, autorización por rol (RBAC) aplicada en el servidor: estudiante, docente, secretario y administrador acceden únicamente a las vistas y operaciones que les corresponden. Tercero, trazabilidad completa: toda modificación sensible deja registro en la tabla de auditoría, con usuario, fecha y valor anterior. Cuarto, protección del canal con

[HTTPS](#), validación de formularios contra falsificación de petición y control de acceso a los recursos estáticos.

Estas medidas responden a los requerimientos no funcionales de seguridad, trazabilidad y disponibilidad del documento de planificación y son condición previa para la puesta en producción del sistema.

Bibliografia

- [1] Django Software Foundation. *Django documentation, Version 4.2.*
<https://docs.djangoproject.com/>
- [2] PostgreSQL Global Development Group. *PostgreSQL Documentation.*
<https://www.postgresql.org/docs/>